



Object Detection

(CAPÍTULO 3 – CV03C)

Domingo Mery 

Ciencia de la Computación
Universidad Católica – Chile
<http://domingomery.ing.uc.cl>



Classification



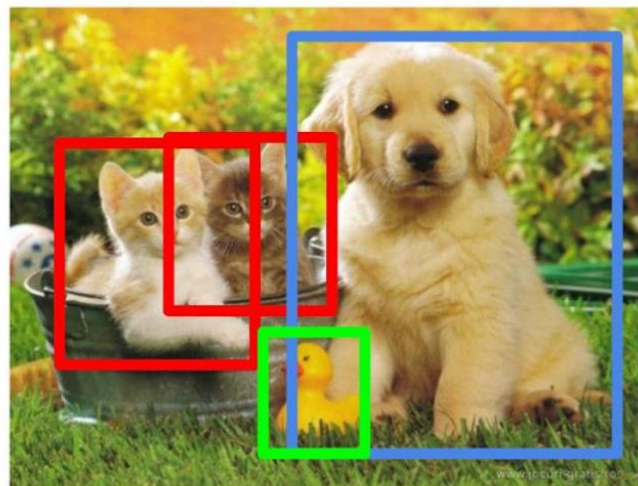
CAT

Classification + Localization



CAT

Object Detection



CAT, DOG, DUCK

Instance Segmentation



CAT, DOG, DUCK

Single object

Multiple objects

Input X-ray Image



Image Classification

'Handgun'

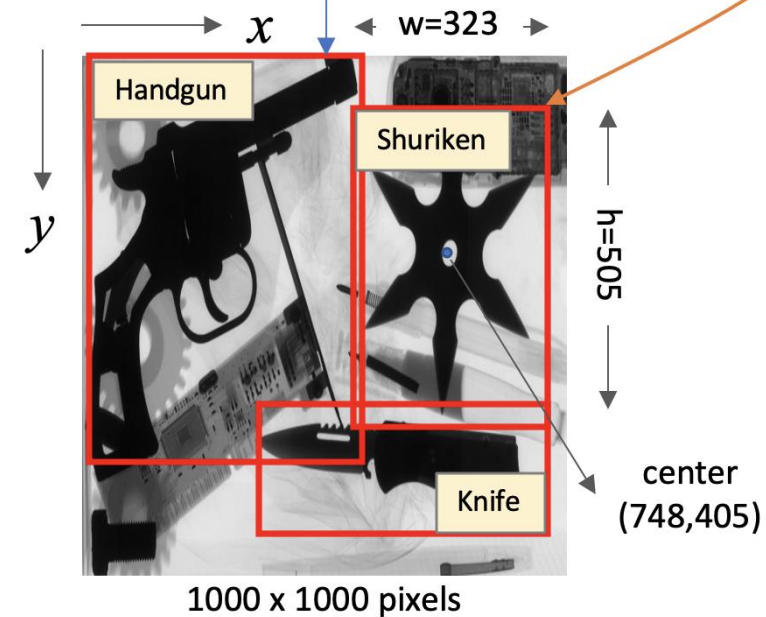
a)

Input X-ray Image



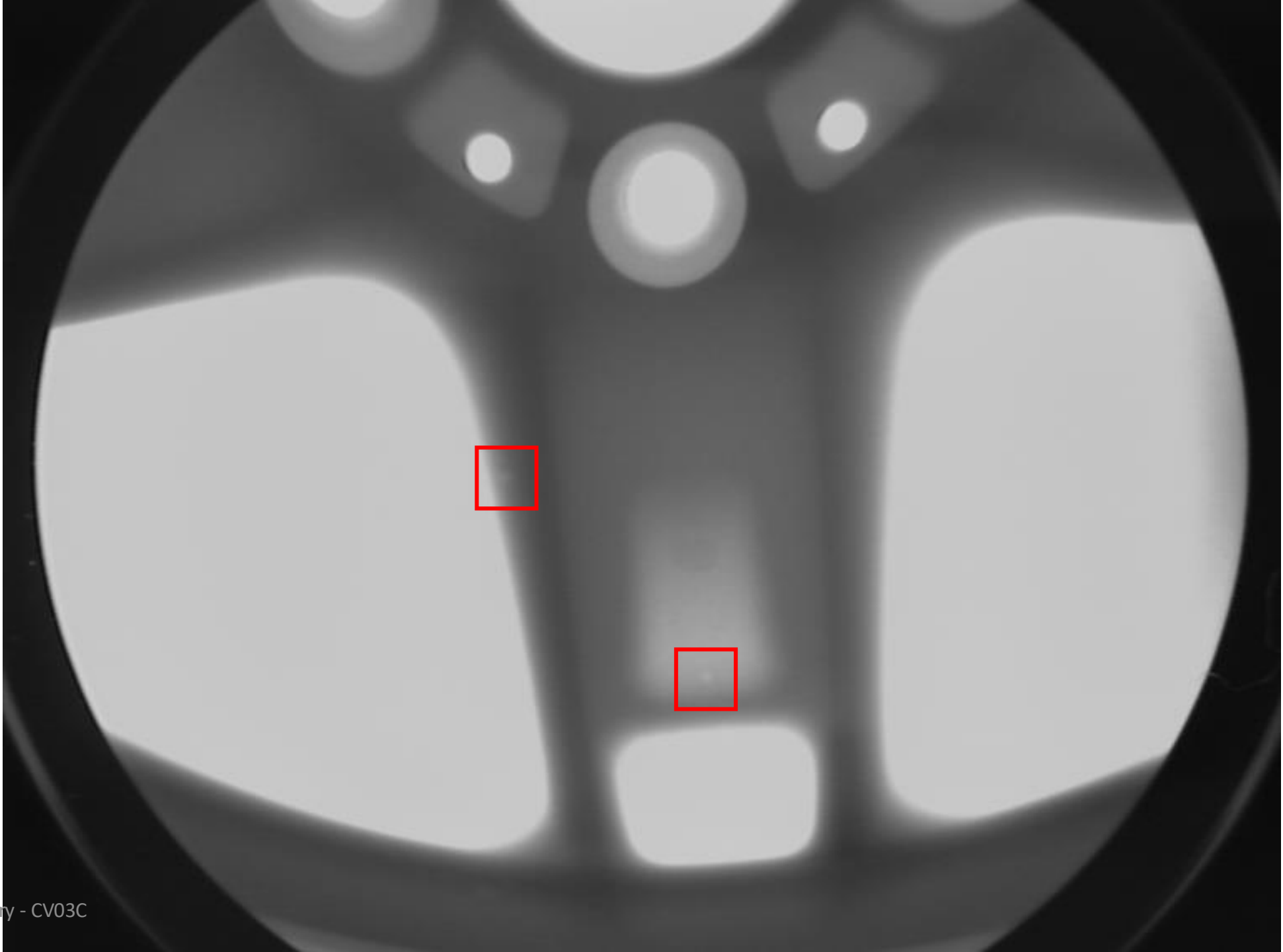
Image Detection

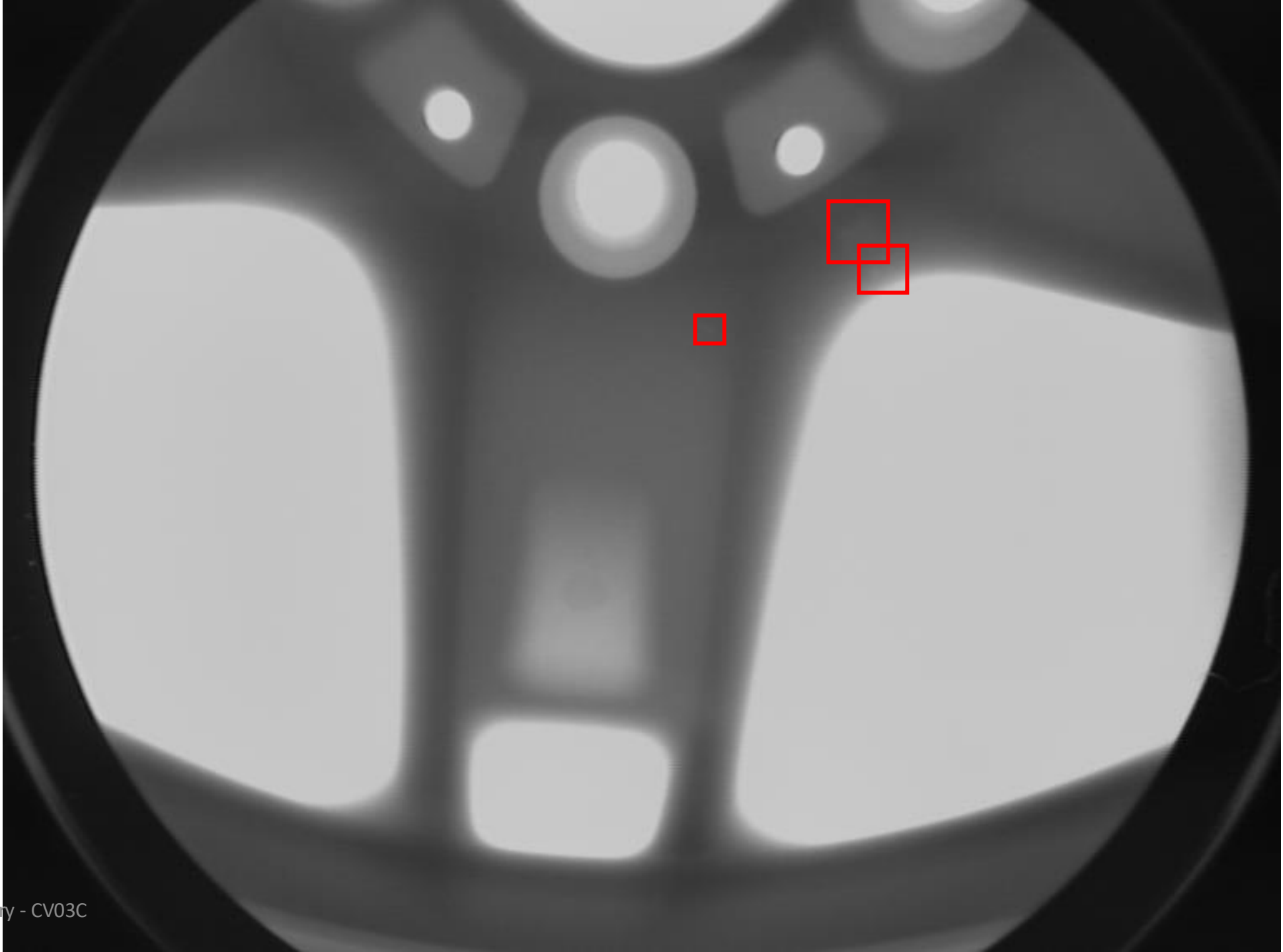
Object	x	y	w	h
Handgun	255	410	510	820
Shuriken	748	405	323	505
Knife	680	850	550	180

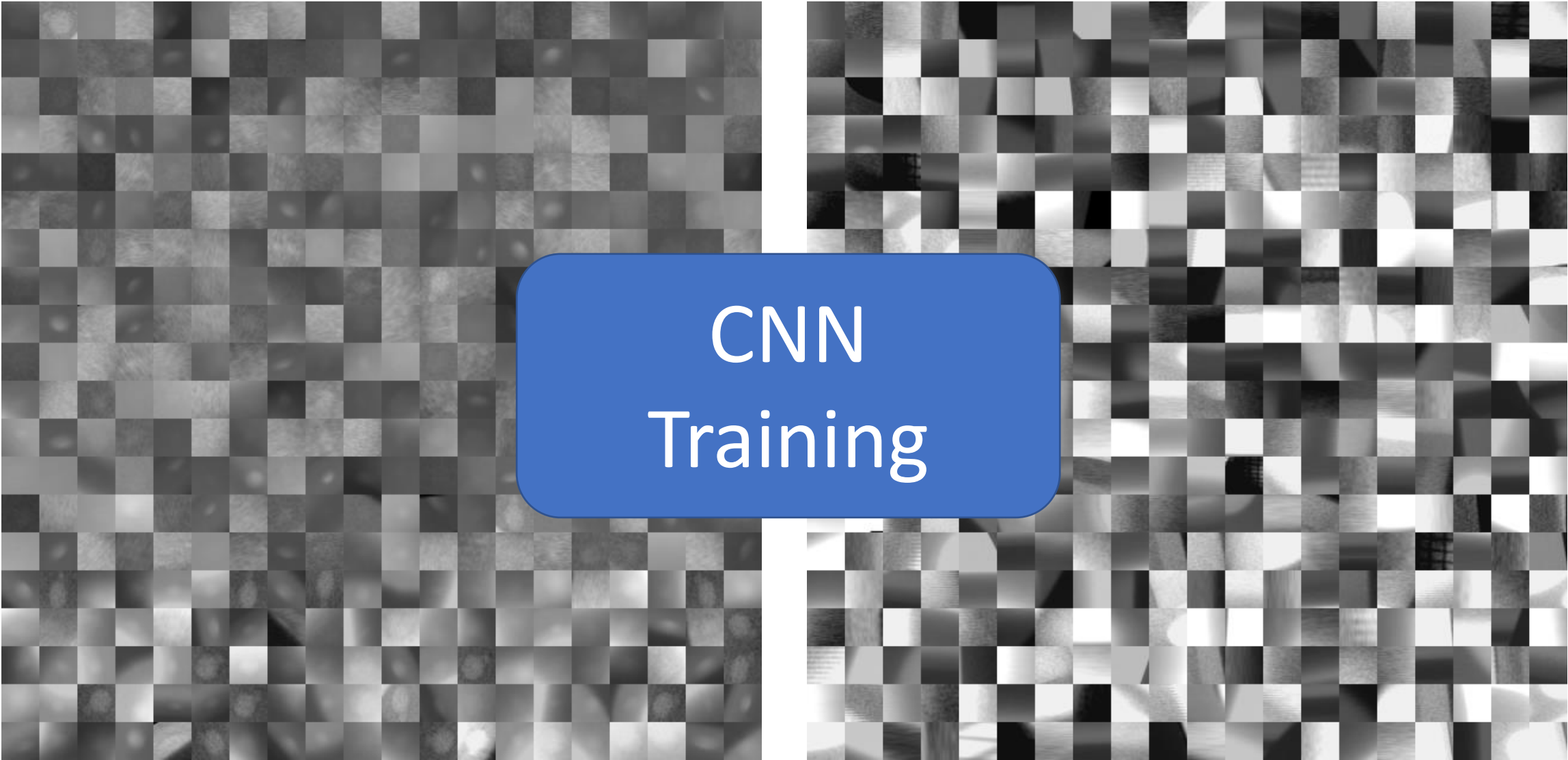


b)

Sliding Windows con CNN







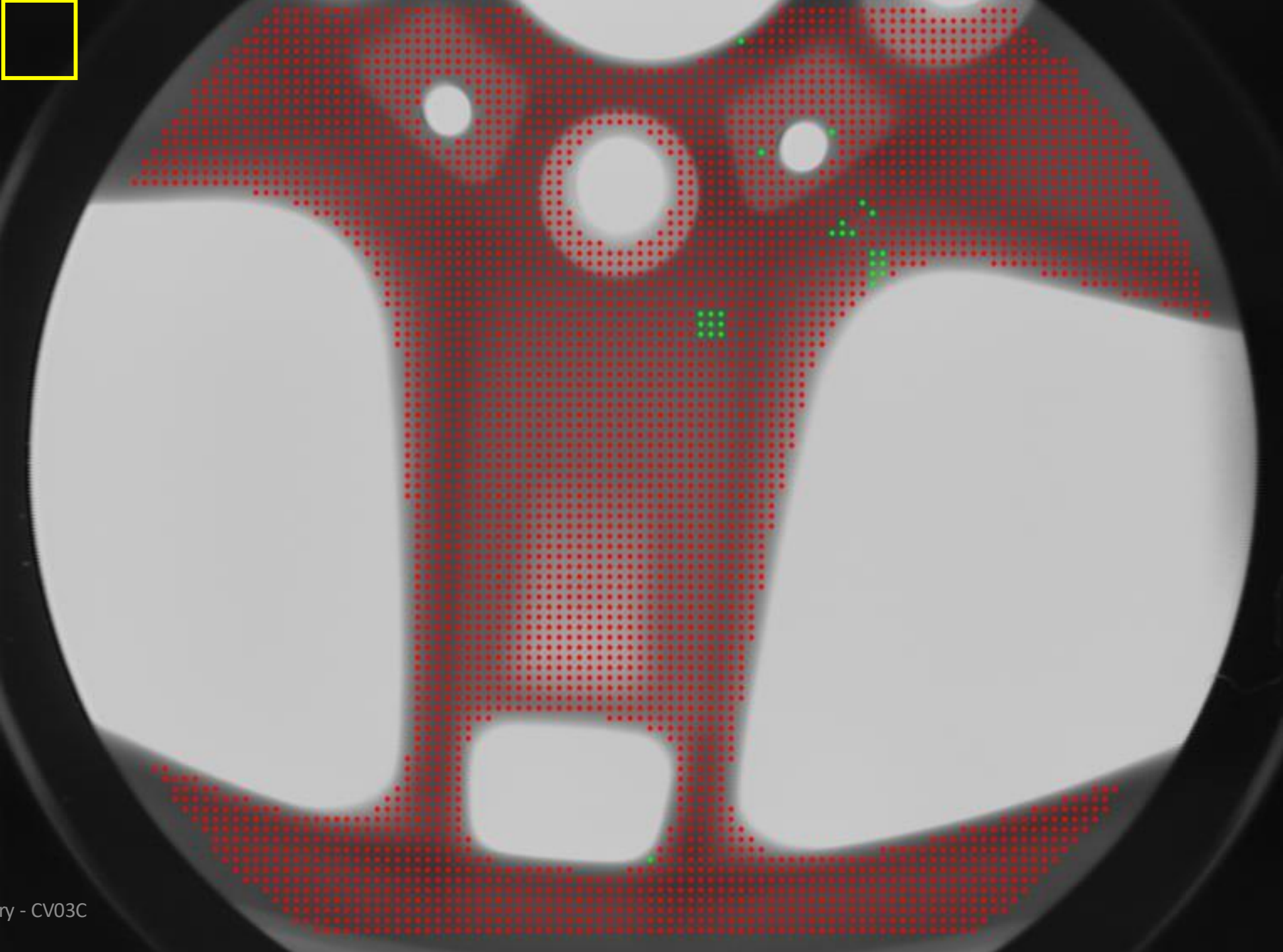
CNN Training

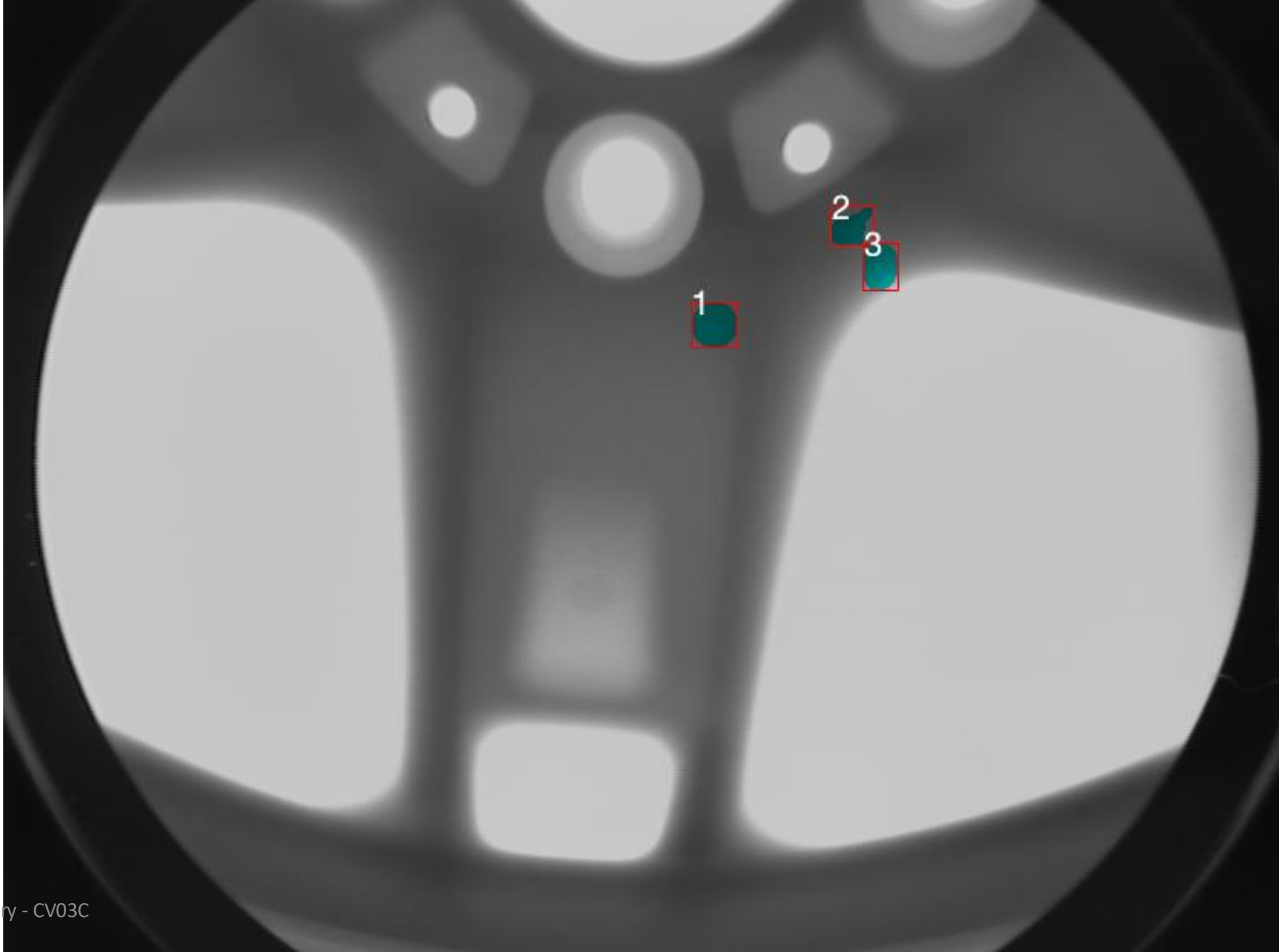
DEFECTS

NO-DEFECTS

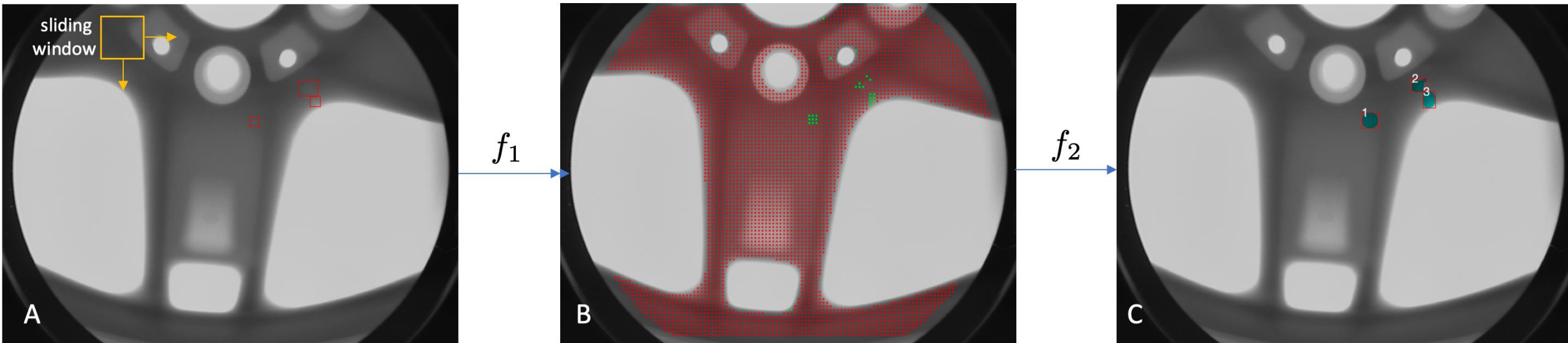


Sliding Windows





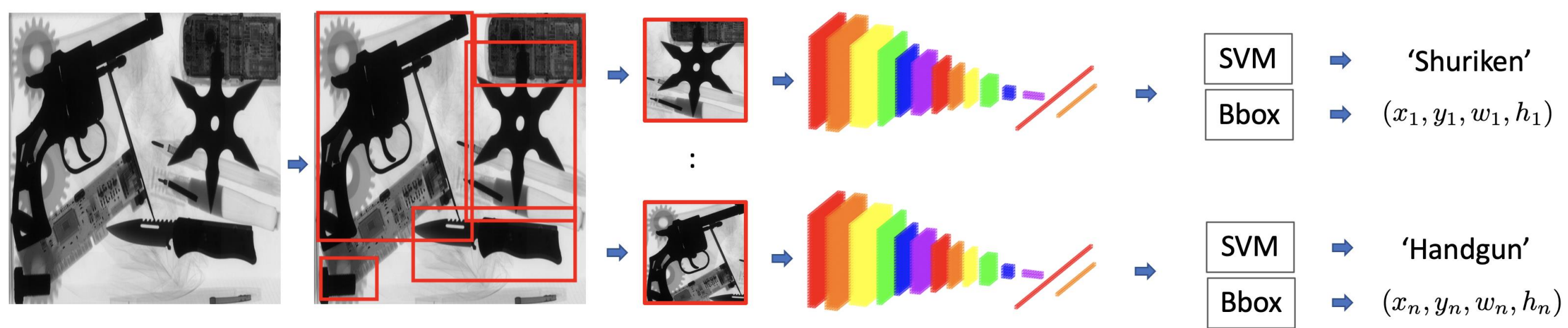
Sliding-Windows



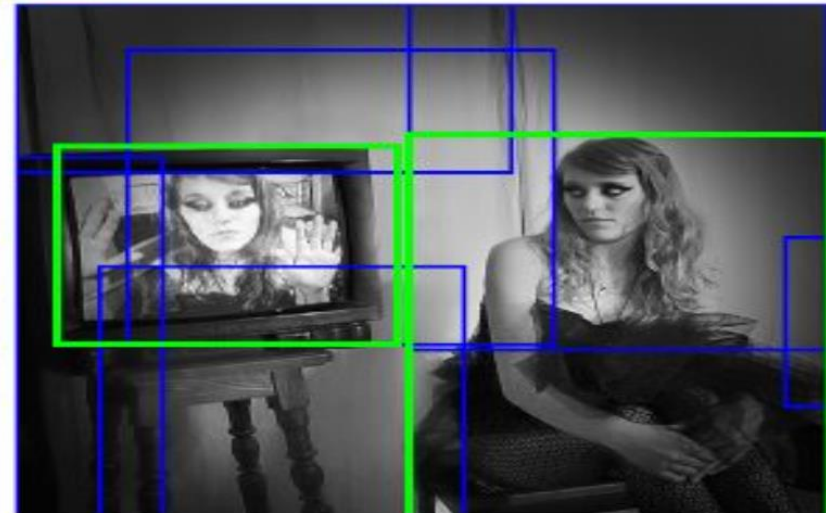
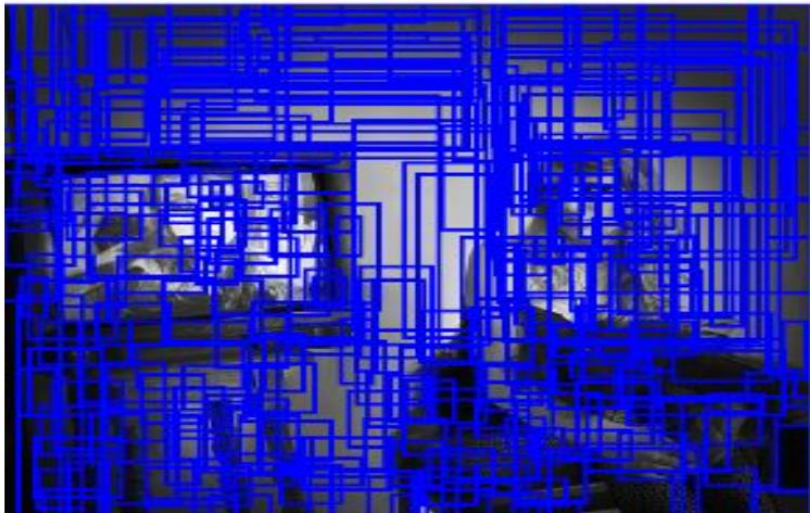
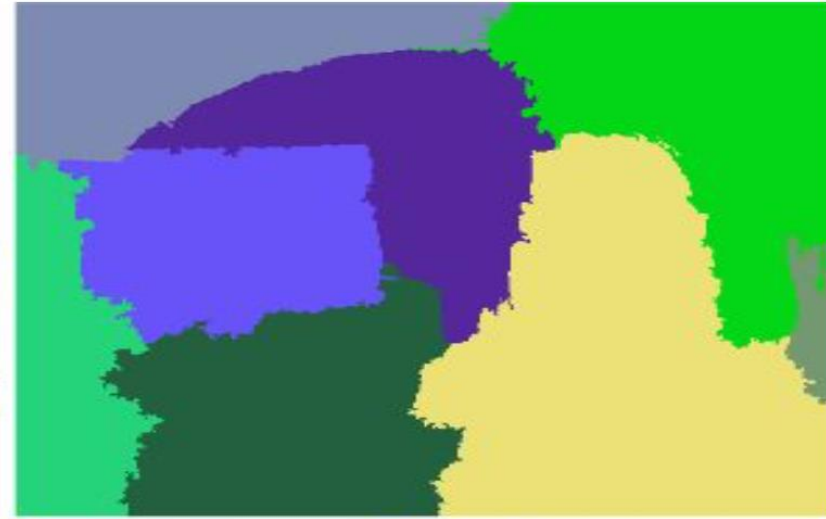
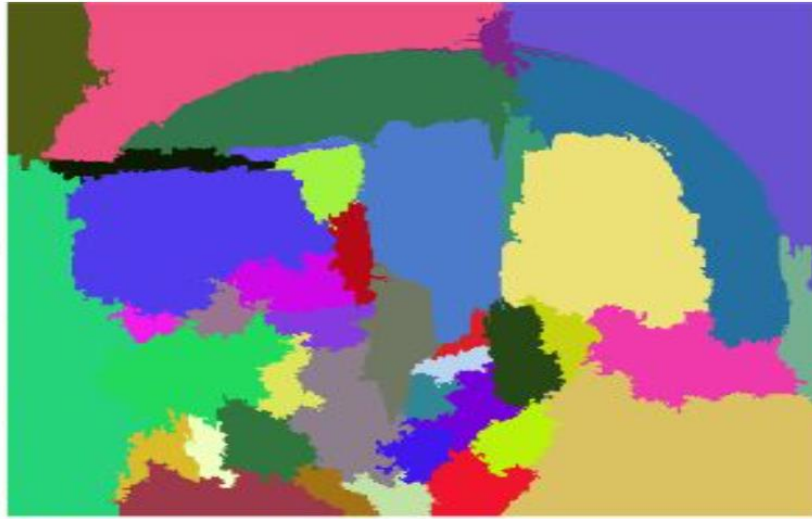
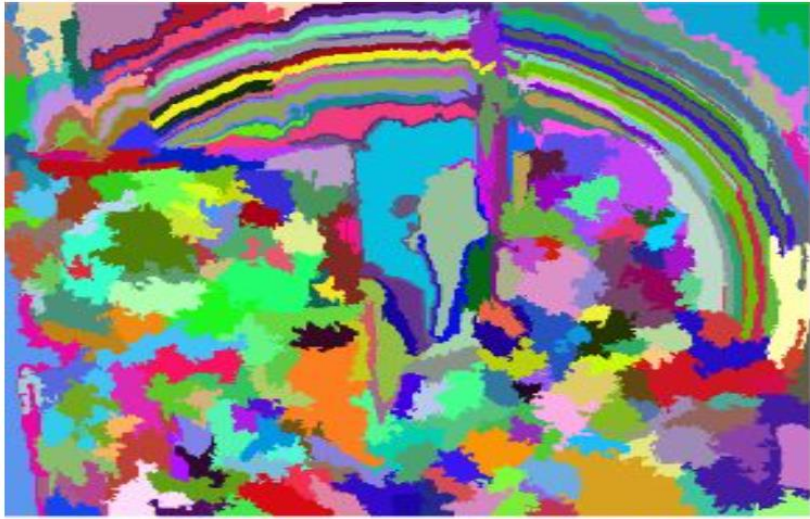
RCNN

RCNN

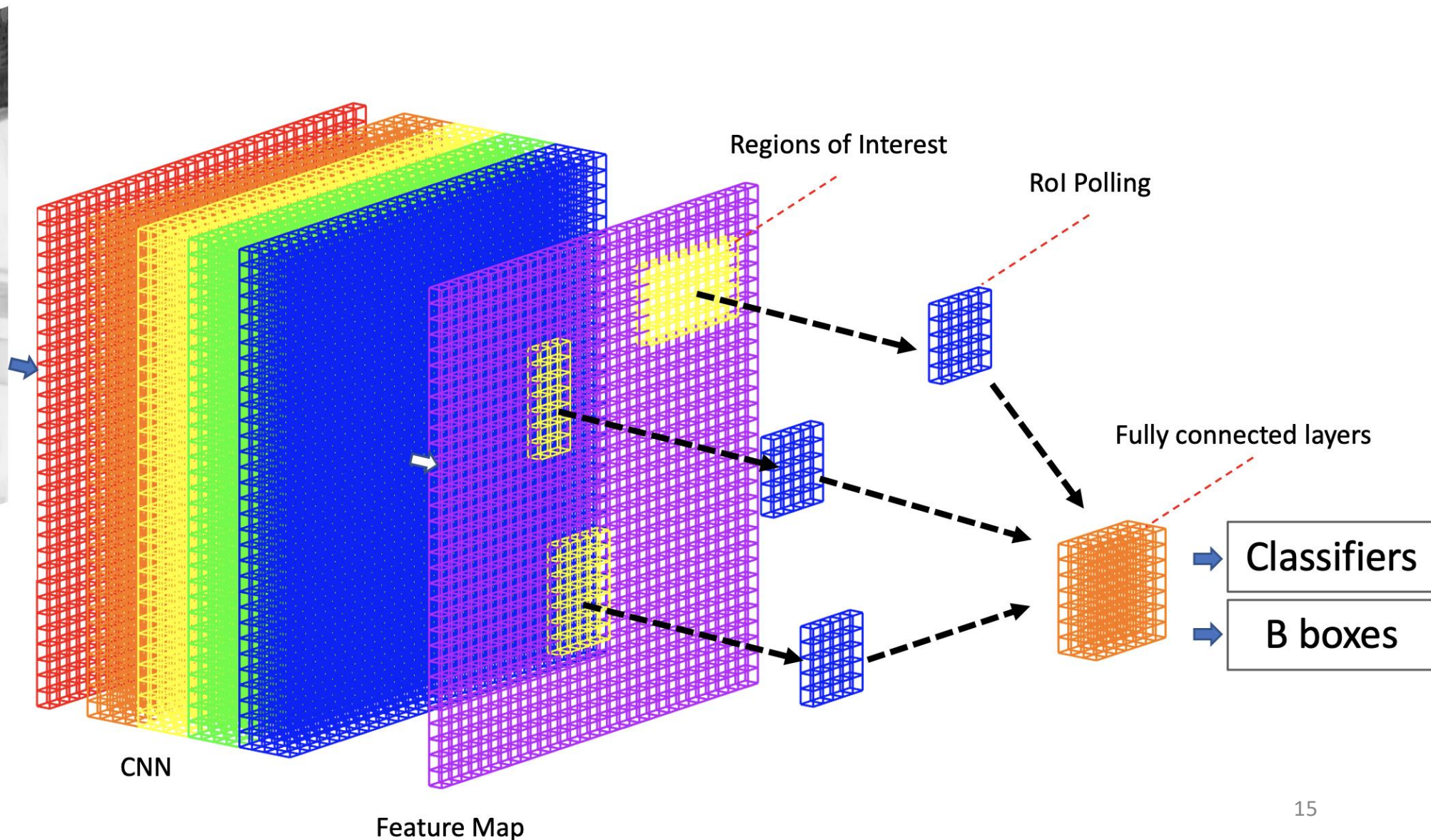
Input Image → Region Proposals → Warped Regions → CNN features → Classify Regions → Output



Selective Search (Uijlings,2012)



Fast-RCNN



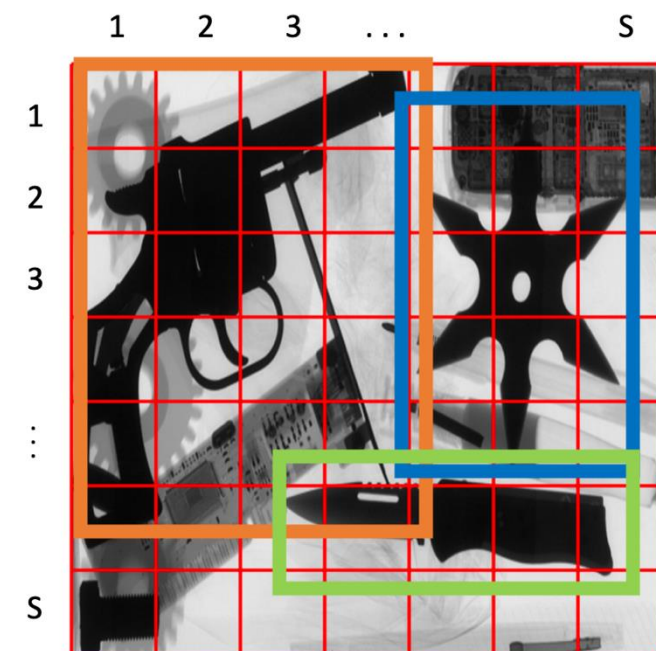
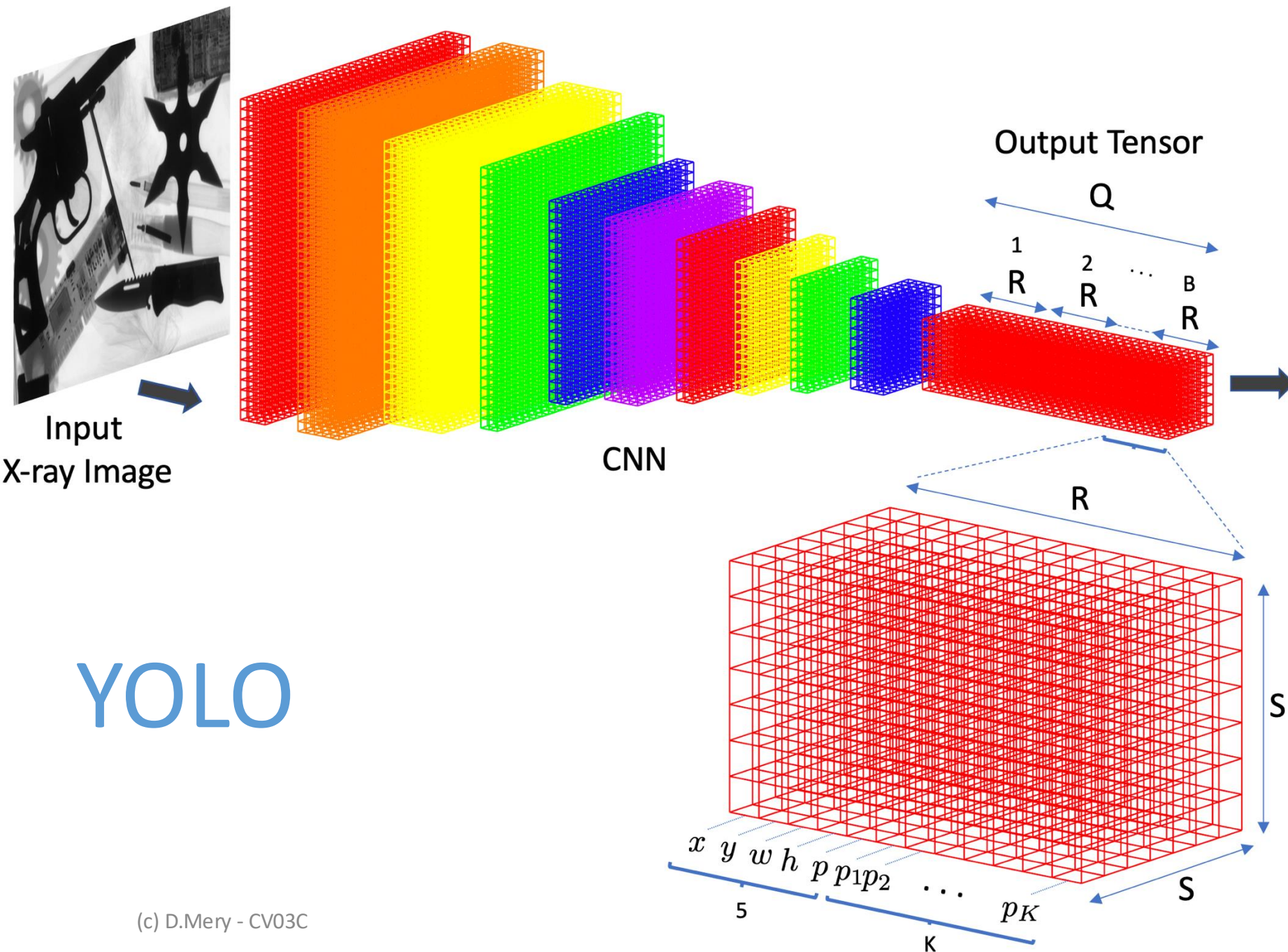
YOLO

The key idea of YOLO is very simple: The input image is divided into a grid of $S \times S$ cells, and for each cell, YOLO can detect B objects. For each detected bounding box, YOLO computes:

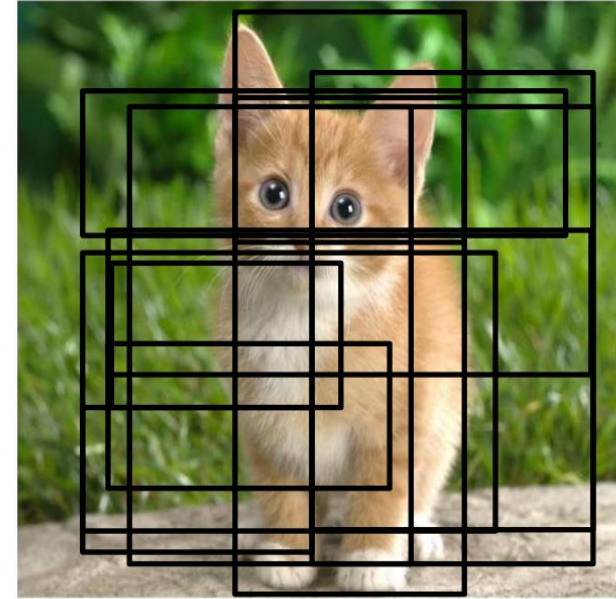
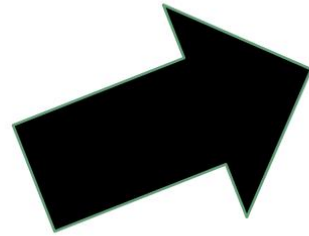
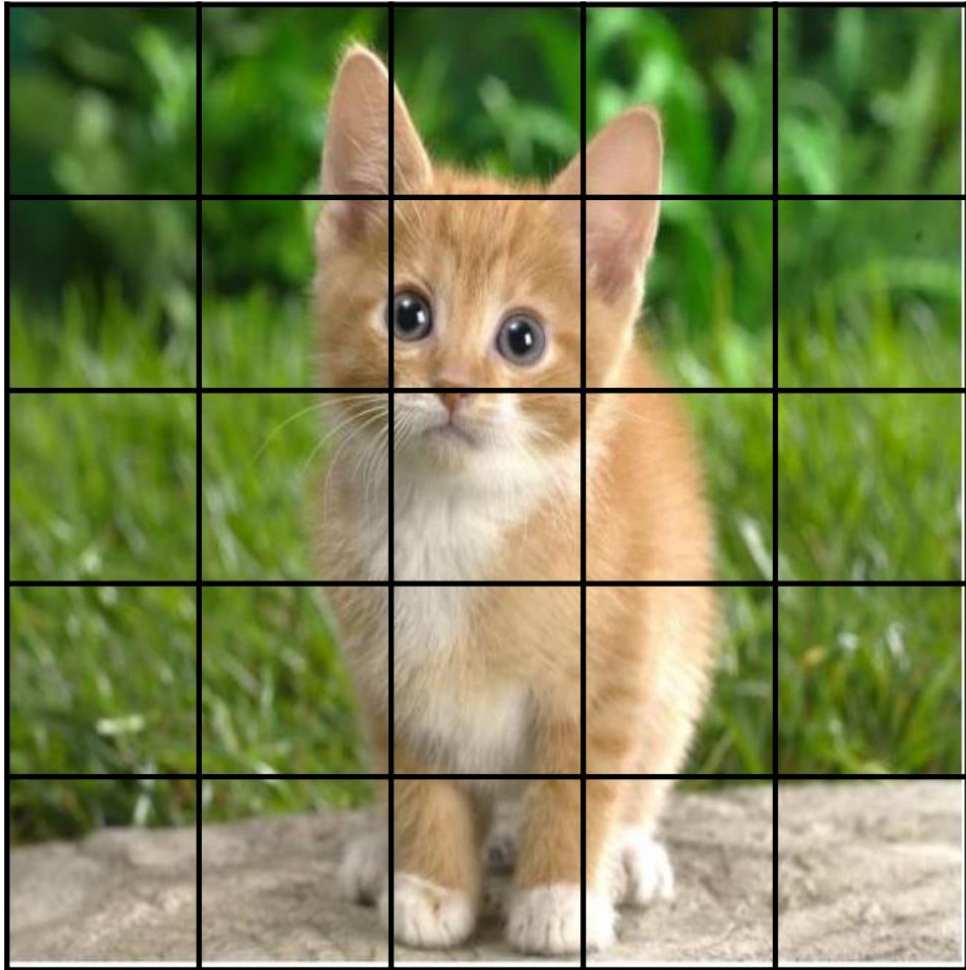
- (x, y, w, h) : variables that define the detected bounding box, i.e., location (x, y) and dimension (width, height),
- p : confidence score that gives the probability that the bounding box encloses an object ($\text{Pr}(\text{Object})$), and
- p_i : for $i = 1 \dots K$: probability distribution over all K possible classes, i.e., p_i is a conditional class probability ($\text{Pr}(\text{Class}_i|\text{Object})$).

That means, for each bounding box, YOLO provides an array of $R = 4 + 1 + K$ elements: $(x, y, w, h, p, p_1, p_2, \dots, p_K)$, as illustrated in Fig. 7.35. In testing stage, an object of class i is detected if $\text{Pr}(\text{Object}) \times \text{Pr}(\text{Class}_i|\text{Object})$ is greater than a threshold.

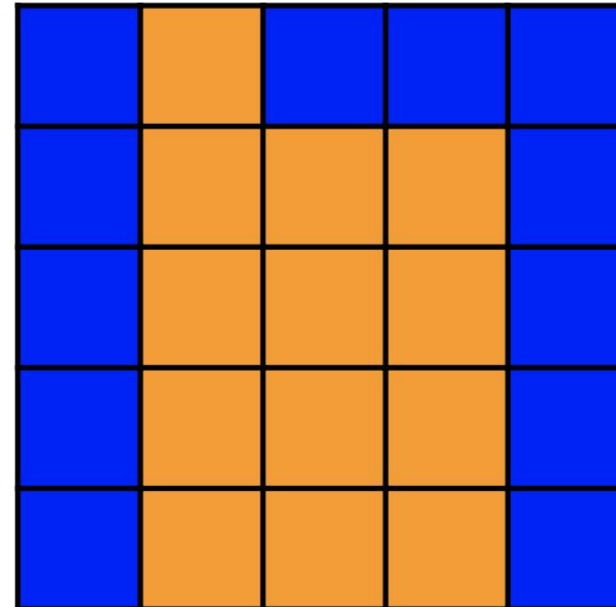
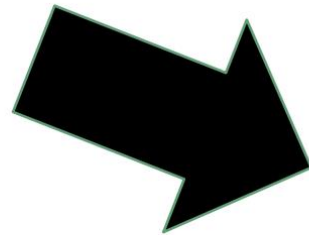
Since in a grid cell, B bounding boxes can be detected, for each cell, an array of $Q = B \times R$ elements is computed.



YOLO



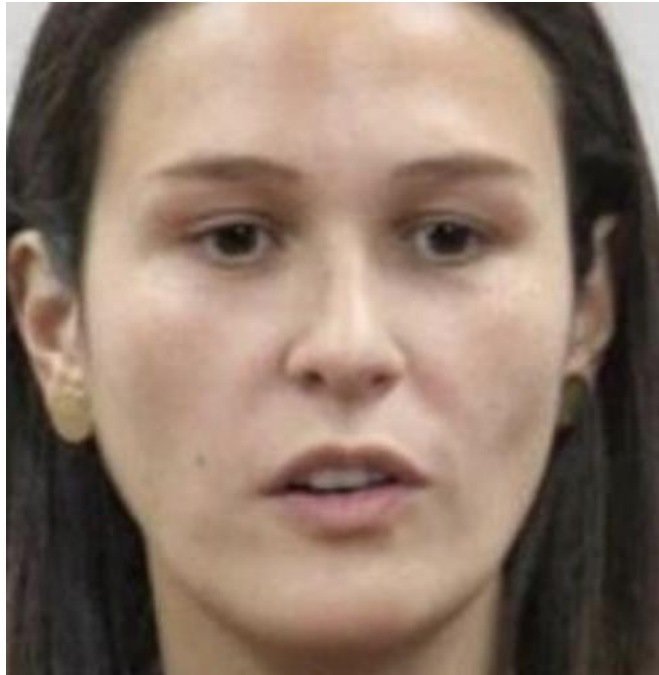
Bounding-box
+ Confidence



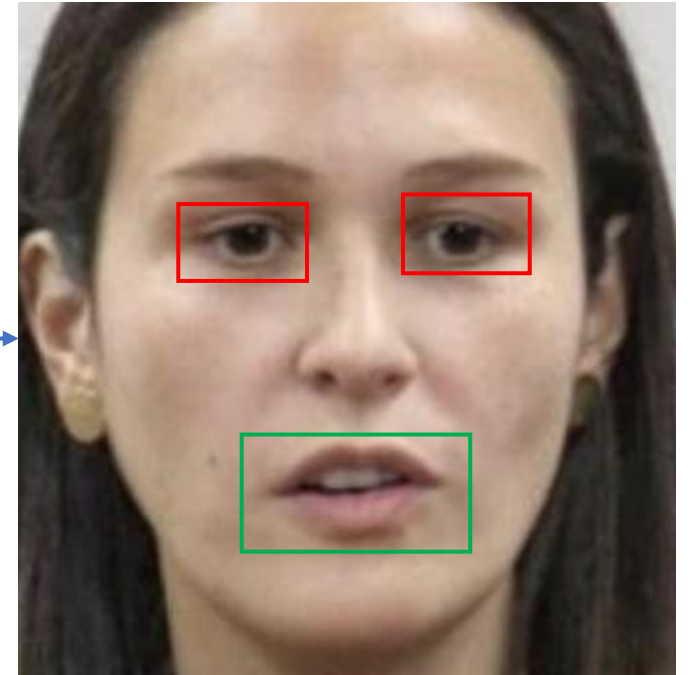
Class Probability
Map

Implementación de YOLOv8

(Ejemplo: Detección de ojos y boca)



Object
Detection



Para la implementación de YOLOv8...

Se necesita:

1. Base de datos (imágenes)
2. Anotaciones (bounding boxes + clases)
3. Formatos adecuados
4. Definición de Train/Val/Test
5. Definición de archivo YAML

1. Base de datos



FM000145_06.jpg



FM000146_01.jpg



FM000146_02.jpg



FM000146_03.jpg



FM000146_04.jpg



FM000146_05.jpg



FM000147_05.jpg



FM000147_06.jpg



FM000148_01.jpg



FM000148_02.jpg



FM000148_03.jpg



FM000148_04.jpg



FM000149_04.jpg



FM000149_05.jpg



FM000149_06.jpg



FM000152_01.jpg

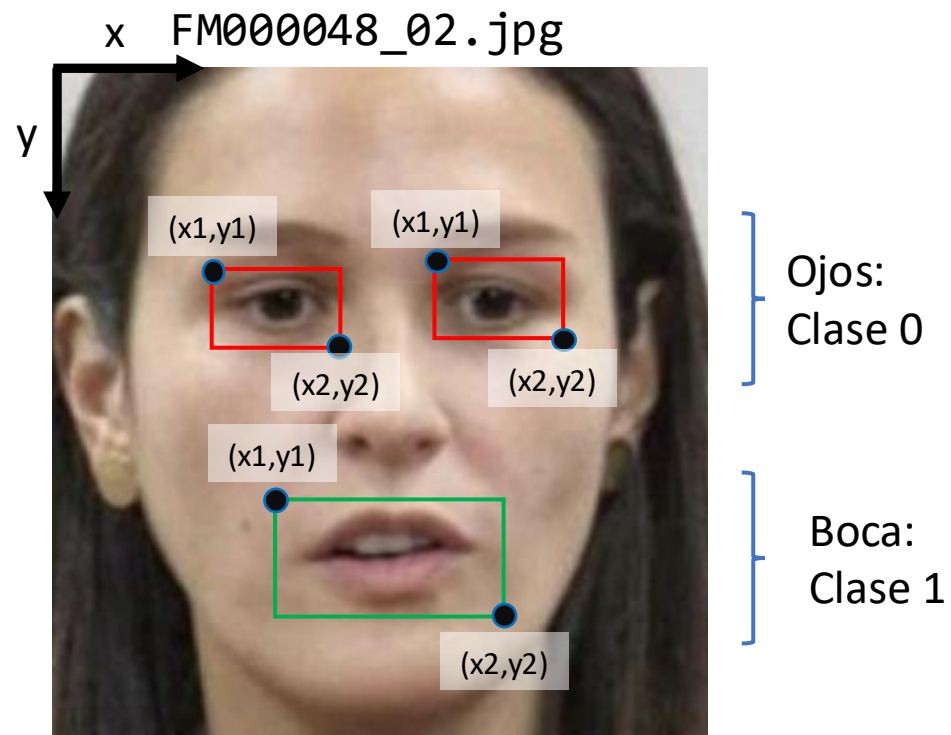


FM000152_02.jpg



FM000152_03.jpg

2. Anotaciones

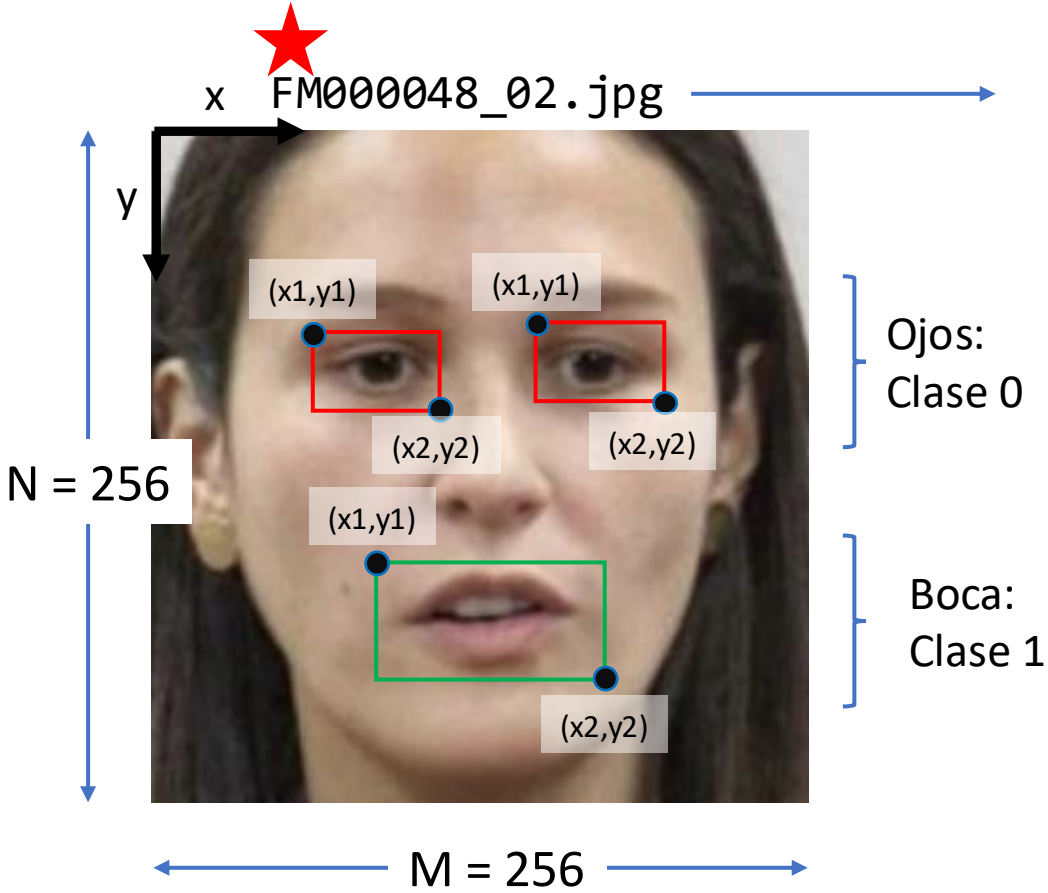


2. Anotaciones

boundingbox.csv

	A	B	C	D	E	F
1	File	x1	x2	y1	y2	c
2	FM000008_01.jpg	74.5544	110.1314	84.0287	101.0438	0
3	FM000008_01.jpg	156.122	188.6185	81.9217	101.4196	0
4	FM000008_01.jpg	91.1291	171.287	157.8583	187.3547	1
5	FM000008_02.jpg	74.5544	110.1314	84.0287	101.0438	0
6	FM000008_02.jpg	156.122	188.6185	81.9217	101.4196	0
7	FM000008_02.jpg	91.1291	171.287	157.8583	187.3547	1
8	FM000008_03.jpg	74.5544	110.1314	84.0287	101.0438	0
9	FM000008_03.jpg	156.122	188.6185	81.9217	101.4196	0
10	FM000008_03.jpg	91.1291	171.287	157.8583	187.3547	1
11	FM000008_04.jpg	74.5544	110.1314	84.0287	101.0438	0
12	FM000008_04.jpg	156.122	188.6185	81.9217	101.4196	0

3. Formato



boundingbox.csv

File	x1	x2	y1	y2	c
FM000048_01.jpg	156.122	188.6185	81.9217	101.4196	0
FM000048_01.jpg	91.1291	171.287	157.8583	187.3547	1
FM000048_02.jpg	74.5544	110.1314	84.0287	101.0438	0
FM000048_02.jpg	156.122	188.6185	81.9217	101.4196	0
FM000048_02.jpg	91.1291	171.287	157.8583	187.3547	1
FM000048_03.jpg	74.5544	110.1314	84.0287	101.0438	0
FM000048_03.jpg	156.122	188.6185	81.9217	101.4196	0

FM000048_02.txt

c	xn	yn	wn	hn
0	0.3607	0.3615	0.1390	0.0665
0	0.6733	0.3581	0.1269	0.0762
1	0.5125	0.6742	0.3131	0.1152

$$x_n = \frac{1}{M} \frac{x_1 + x_2}{2} \quad y_n = \frac{1}{N} \frac{y_1 + y_2}{2} \quad w_n = \frac{x_2 - x_1}{M} \quad h_n = \frac{y_2 - y_1}{N}$$

★ Por cada imagen con objetos <nombre_imagen>.jpg se necesita un archivo de texto <nombre_imagen>.txt

4. Definición de Train/Val/Test

```
data
|-----images
|       |-----train — <trainimage01>.jpg
|       |               <trainimage02>.jpg
|       |               :
|       |-----val   — <valimage01>.jpg
|       |               <valimage02>.jpg
|       |               :
|       |-----test  — <testimage01>.jpg
|       |               <testimage02>.jpg
|       |               :
|-----labels
|       |-----train — <trainimage01>.txt
|       |               <trainimage02>.txt
|       |               :
|       |-----val   — <valimage01>.txt
|       |               <valimage02>.txt
|       |               :
|       |-----test  — <testimage01>.txt
|       |               <testimage02>.txt
|       |               :
|       |               28
|       |               :
```

5. Definición de archivo YAML

```
data/data.yaml
```

```
train: ../data/images/train  
val:   ../data/images/val  
test:  ../data/images/test  
nc:    2  
names: [ 'eye', 'mouth' ]
```

(archivo de texto: data.yaml)

Ahora estamos listos para entrenar y probar YOLOv8!

Training

```
YOLO_MODEL      = 'yolov8s'  
EPOCHS          = 100  
TRAIN_WEIGHTS   = YOLO_MODEL + '.pt'  
BATCH           = 8  
IMG_SIZE        = 256  
YAMLFILE        = 'data/data.yaml'
```

Starting training for 100 epochs...

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
1/100	0.543G	3.021	3.464	1.748	16	256: 100% 16/16 4.7it/s 3.4s
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 1/1 1.1it/
	all	14	35	0.206	0.464	0.242 0.0535

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
2/100	0.607G	2.23	1.699	1.313	19	256: 100% 16/16 14.8it/s 1.1s
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 1/1 17.9it
	all	14	35	0.367	0.625	0.455 0.141

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
3/100	0.664G	2.153	1.394	1.318	17	256: 100% 16/16 15.8it/s 1.0s
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 1/1 18.4it
	all	14	35	0.229	0.857	0.305 0.101

:

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
100/100	7.06G	0.9323	0.6473	0.902	12	256: 100% 16/16 15.8it/s 1.0s
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 1/1 15.7it
	all	14	35	0.704	0.714	0.634 0.253

100 epochs completed in 0.040 hours.

Optimizer stripped from /content/runs/detect/train/weights/last.pt, 22.5MB

Optimizer stripped from /content/runs/detect/train/weights/best.pt, 22.5MB

← Pesos del entrenamiento

Testing

Pesos del entrenamiento

```
TEST_WEIGHTS = '/content/runs/detect/train/weights/best.pt'  
PATH_TEST    = '/content/data/images/test'
```

```
image 44/59 /content/data/images/test/FM000155_03.jpg: 256x256 2 eyes, 1 mouth, 5.6ms  
image 45/59 /content/data/images/test/FM000155_04.jpg: 256x256 2 eyes, 6.3ms  
image 46/59 /content/data/images/test/FM000155_05.jpg: 256x256 4 eyes, 5.6ms  
image 47/59 /content/data/images/test/FM000155_06.jpg: 256x256 2 eyes, 6.0ms  
image 48/59 /content/data/images/test/FM000163_01.jpg: 256x256 3 eyes, 1 mouth, 5.7ms  
image 49/59 /content/data/images/test/FM000163_02.jpg: 256x256 2 eyes, 1 mouth, 5.7ms  
image 50/59 /content/data/images/test/FM000163_03.jpg: 256x256 3 eyes, 1 mouth, 5.5ms  
image 51/59 /content/data/images/test/FM000163_04.jpg: 256x256 1 eye, 5.7ms  
image 52/59 /content/data/images/test/FM000163_05.jpg: 256x256 1 eye, 5.4ms  
image 53/59 /content/data/images/test/FM000163_06.jpg: 256x256 1 eye, 5.8ms  
image 54/59 /content/data/images/test/FM000164_01.jpg: 256x256 3 eyes, 1 mouth, 5.6ms  
image 55/59 /content/data/images/test/FM000164_02.jpg: 256x256 2 eyes, 1 mouth, 5.5ms  
image 56/59 /content/data/images/test/FM000164_03.jpg: 256x256 1 eye, 1 mouth, 5.6ms  
image 57/59 /content/data/images/test/FM000164_04.jpg: 256x256 2 eyes, 5.4ms  
image 58/59 /content/data/images/test/FM000164_05.jpg: 256x256 2 eyes, 5.8ms  
image 59/59 /content/data/images/test/FM000164_06.jpg: 256x256 3 eyes, 5.5ms  
Speed: 0.5ms preprocess, 6.6ms inference, 1.2ms postprocess per image at shape (1, 3, 256, 256)  
Results saved to /content/runs/detect/predict
```

Resultados



Ejemplo con pesos pre-entrenados

Original





Ejercicio: Detección de Mascarilla

Detección de Mascarilla



Object
Detection



Tracking



METRICS

(es necesario medir la calidad de la detección)

$$\textbf{Precision} = \frac{\textbf{TP}}{\textbf{TP} + \textbf{FP}}$$

How many of the predictions were defects.

$$\textbf{Recall} = \frac{\textbf{TP}}{\textbf{TP} + \textbf{FN}}$$

How many of the defects were predicted.

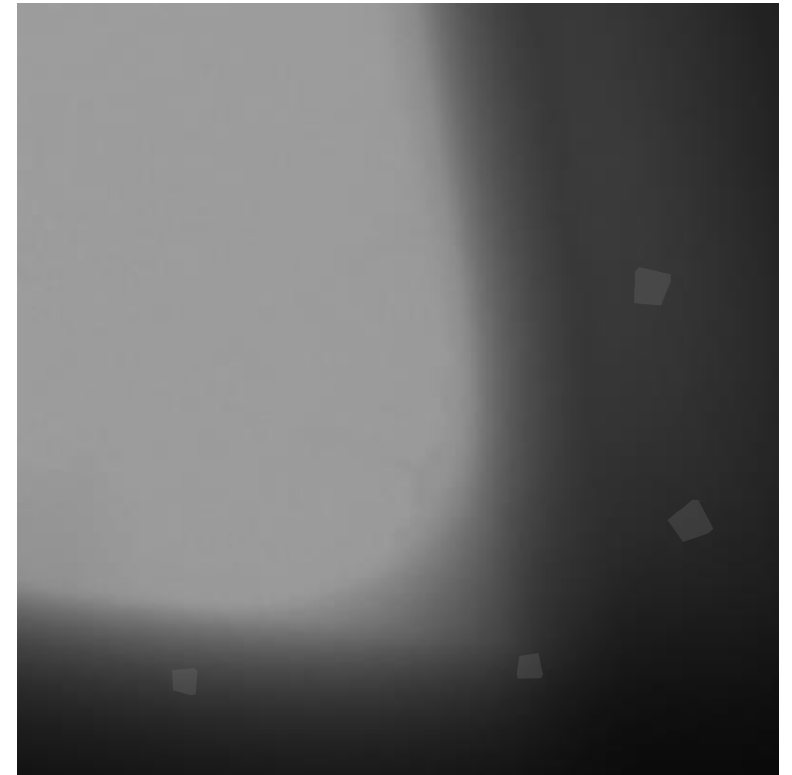
$$\textbf{Precision} = \frac{\textbf{TP}}{\textbf{TP} + \textbf{FP}}$$

How many of the predictions were defects.

$$\textbf{Recall} = \frac{\textbf{TP}}{\textbf{TP} + \textbf{FN}}$$

How many of the defects were predicted.

Example



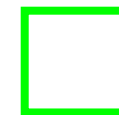
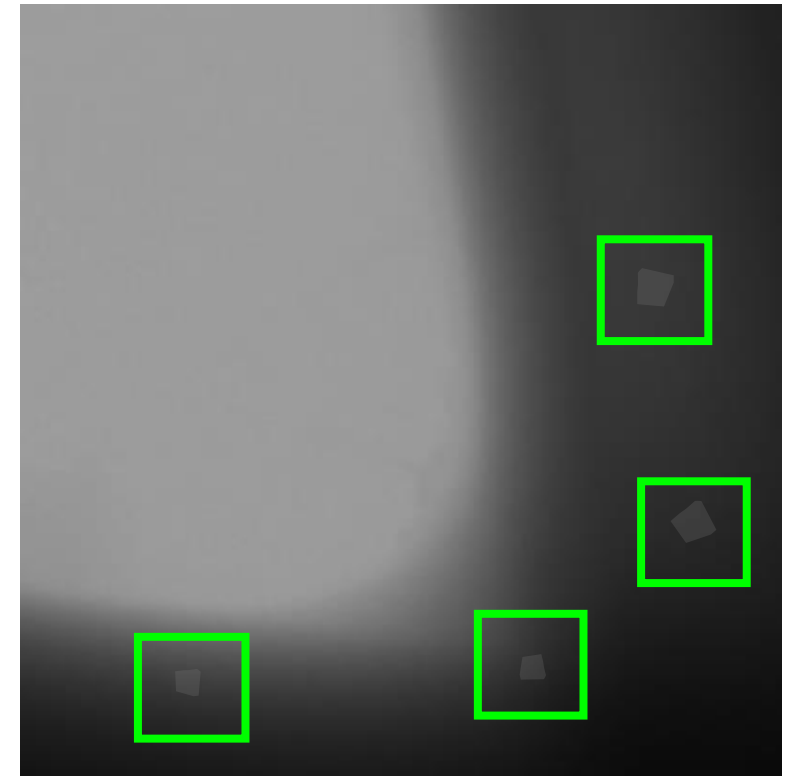
$$\textbf{Precision} = \frac{TP}{TP + FP}$$

How many of the predictions were defects.

$$\textbf{Recall} = \frac{TP}{TP + FN}$$

How many of the defects were predicted.

Example



Ground Truth

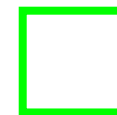
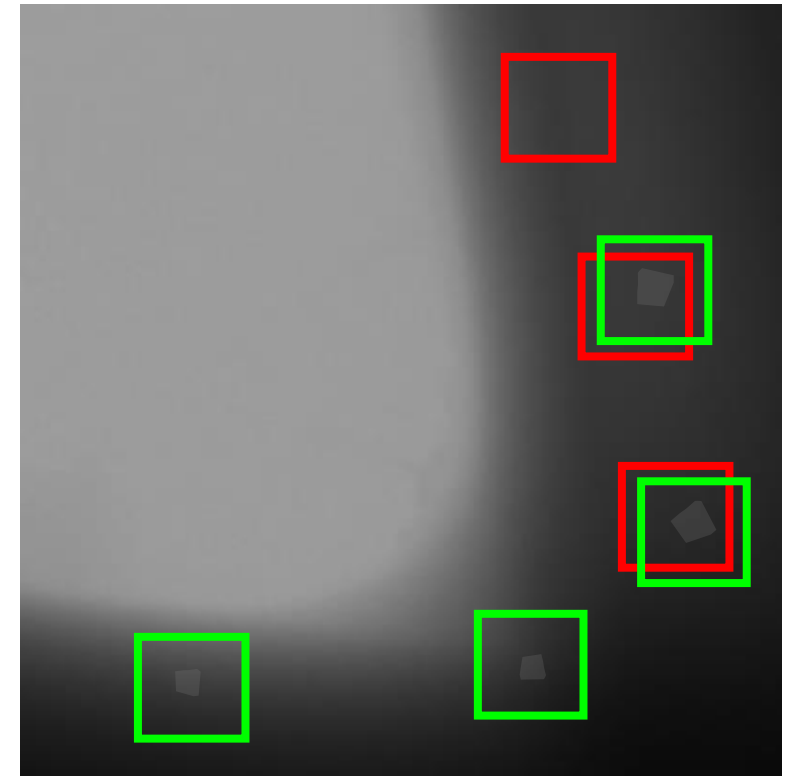
$$\textit{Precision} = \frac{TP}{TP + FP}$$

How many of the predictions were defects.

$$\textit{Recall} = \frac{TP}{TP + FN}$$

How many of the defects were predicted.

Example



Ground Truth



Prediction

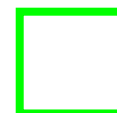
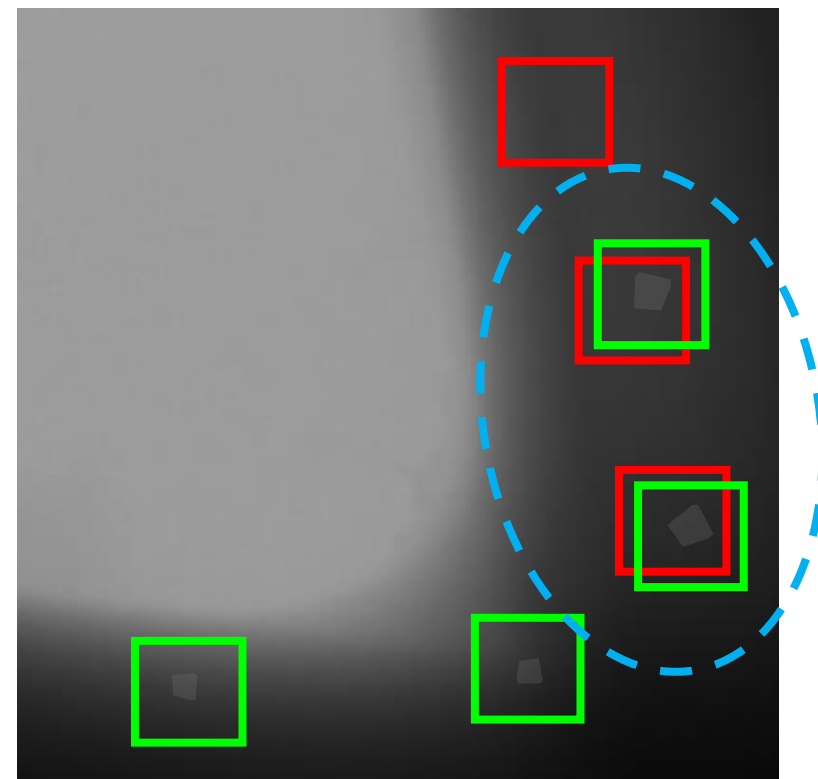
$$\textit{Precision} = \frac{\textit{TP}}{\textit{TP} + \textit{FP}}$$

How many of the predictions were defects.

$$\textit{Recall} = \frac{\textit{TP}}{\textit{TP} + \textit{FN}}$$

How many of the defects were predicted.

Example



Ground Truth



Prediction

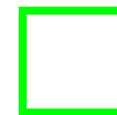
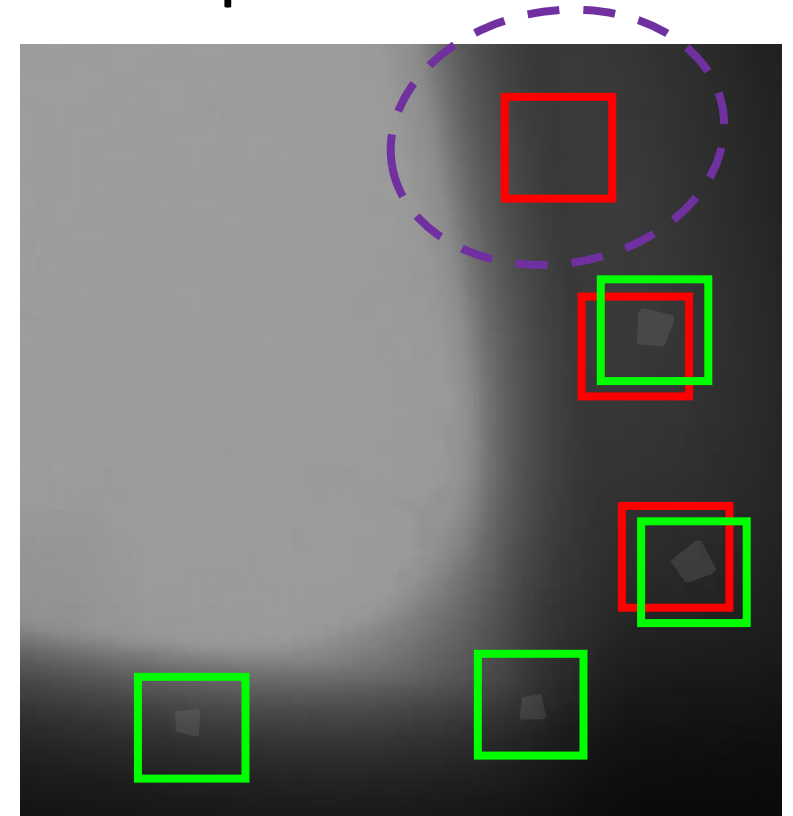
$$\textit{Precision} = \frac{TP}{TP + FP}$$

How many of the predictions were defects.

$$\textit{Recall} = \frac{TP}{TP + FN}$$

How many of the defects were predicted.

Example



Ground Truth



Prediction

$$\text{Precision} = \frac{TP}{TP + FP}$$

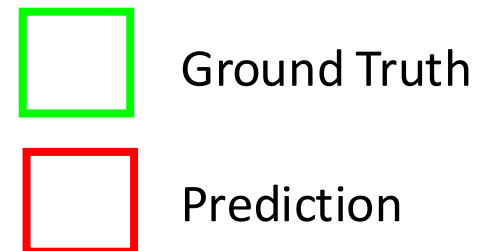
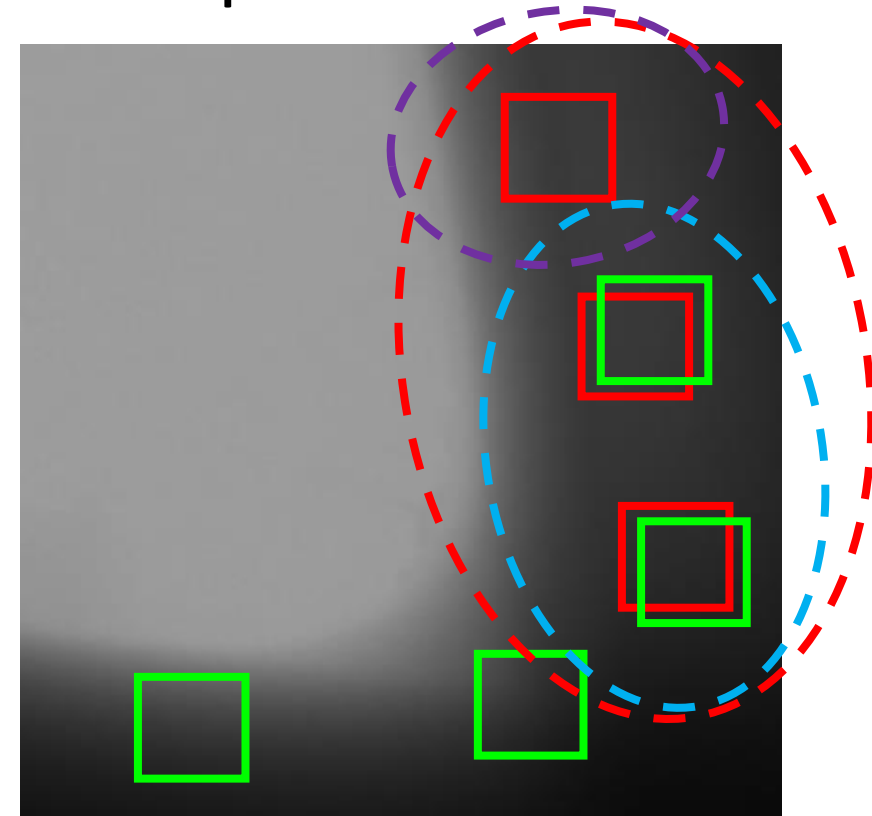
2/3

How many of the predictions were defects.

$$\text{Recall} = \frac{TP}{TP + FN}$$

How many of the defects were predicted.

Example



$$\text{Precision} = \frac{TP}{TP + FP}$$

2/3

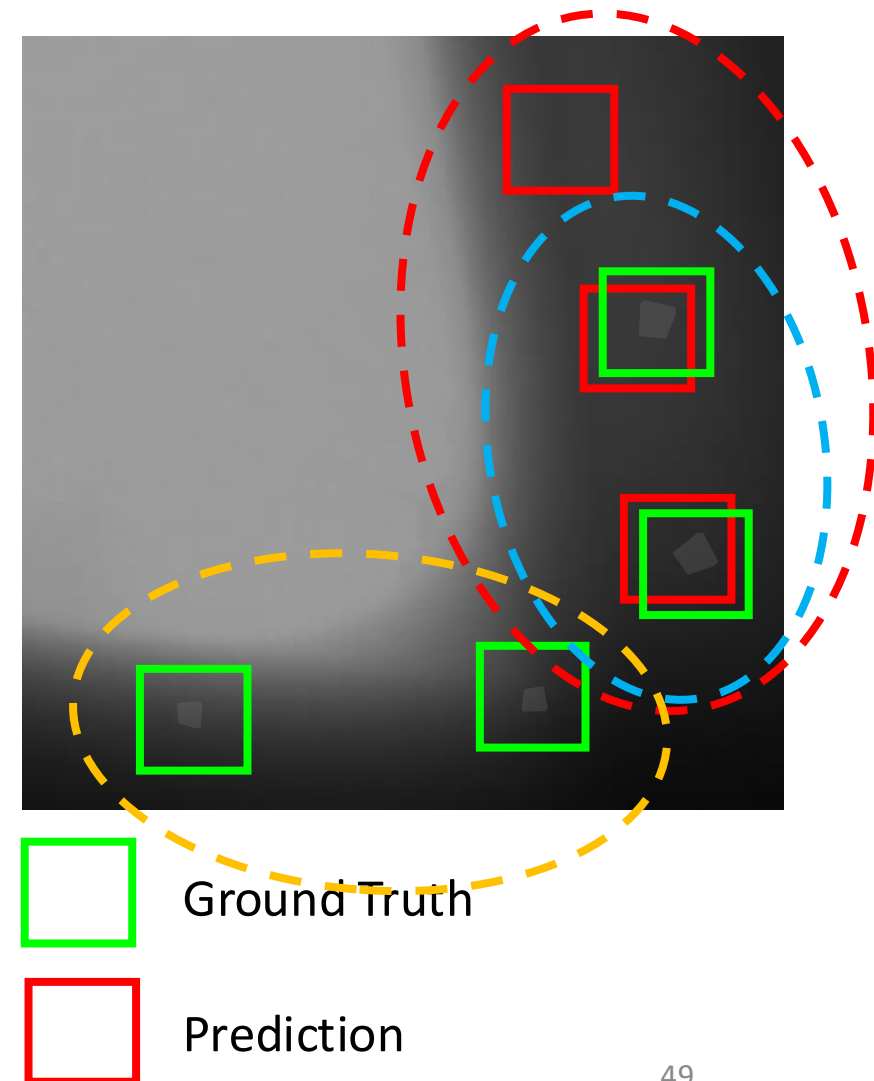
How many of the predictions were defects.

$$\text{Recall} = \frac{TP}{TP + FN}$$

2/4

How many of the defects were predicted.

Example



$$\text{Precision} = \frac{TP}{TP + FP}$$

2/3

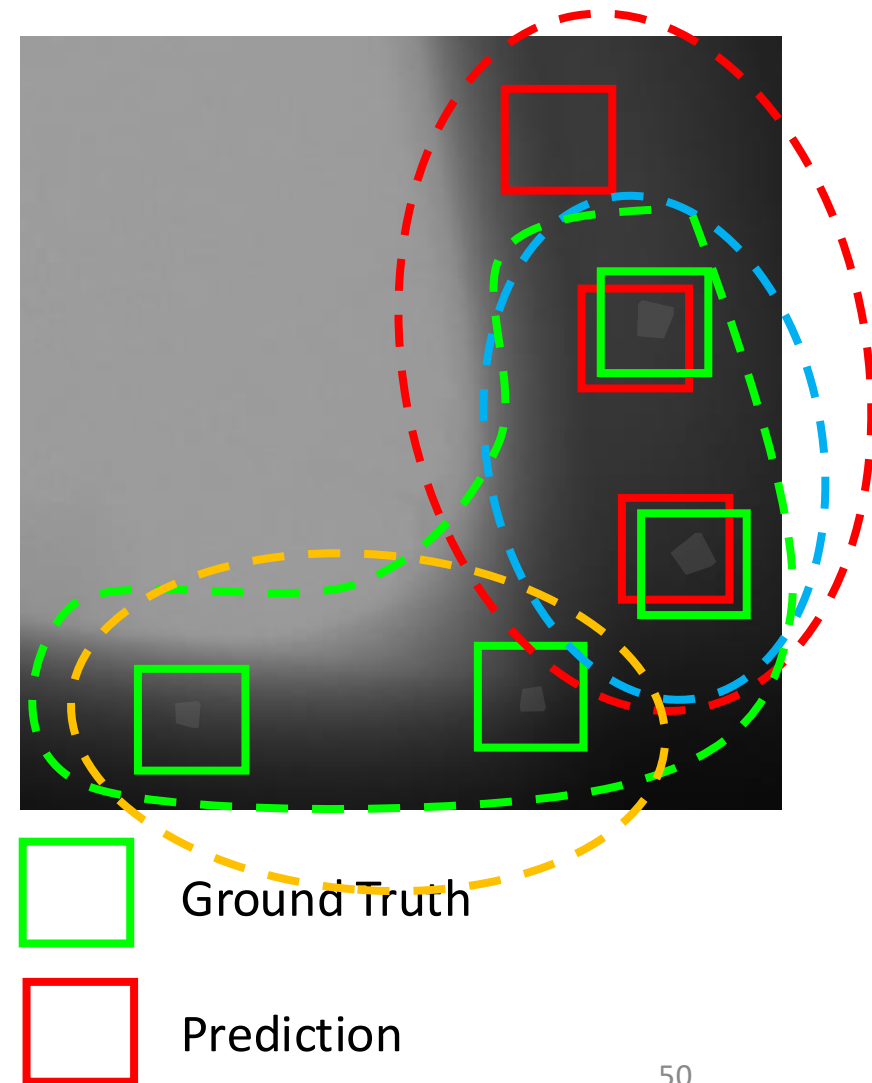
How many of the predictions were defects.

$$\text{Recall} = \frac{TP}{TP + FN}$$

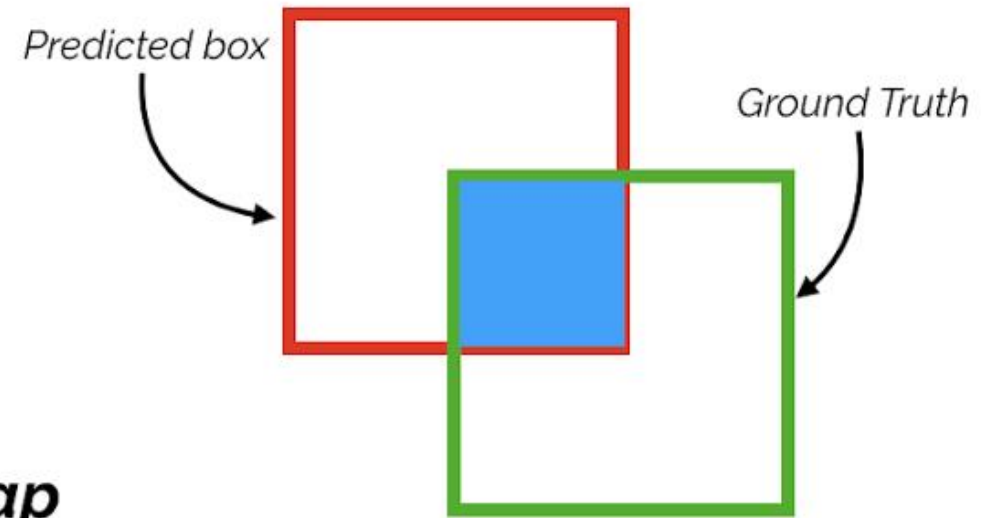
2/4

How many of the defects were predicted.

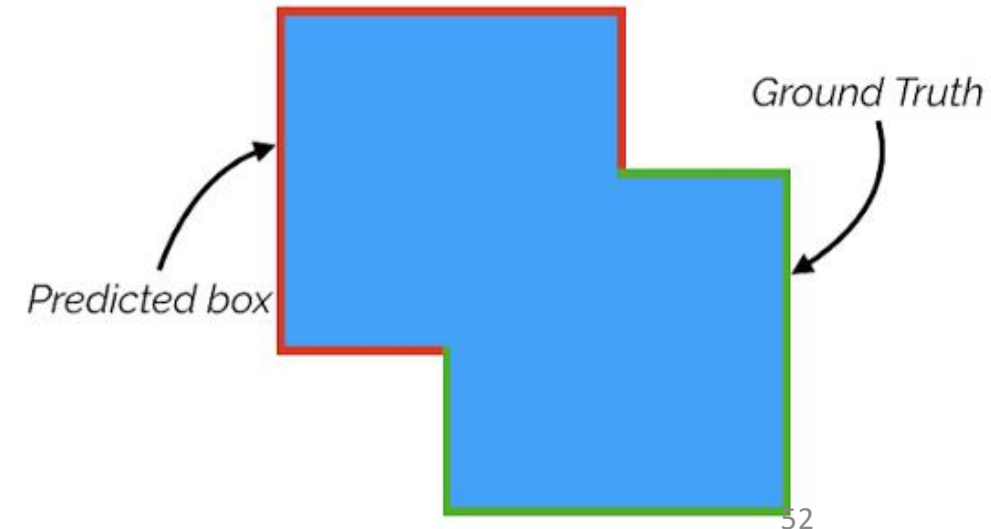
Example



$$\text{Intersection over Union (IoU)} = \frac{\text{Area of Overlap}}{\text{Area of Union}} = \frac{\text{Area of Overlap}}{\text{Area of Predicted box} + \text{Area of Ground Truth} - \text{Area of Overlap}}$$



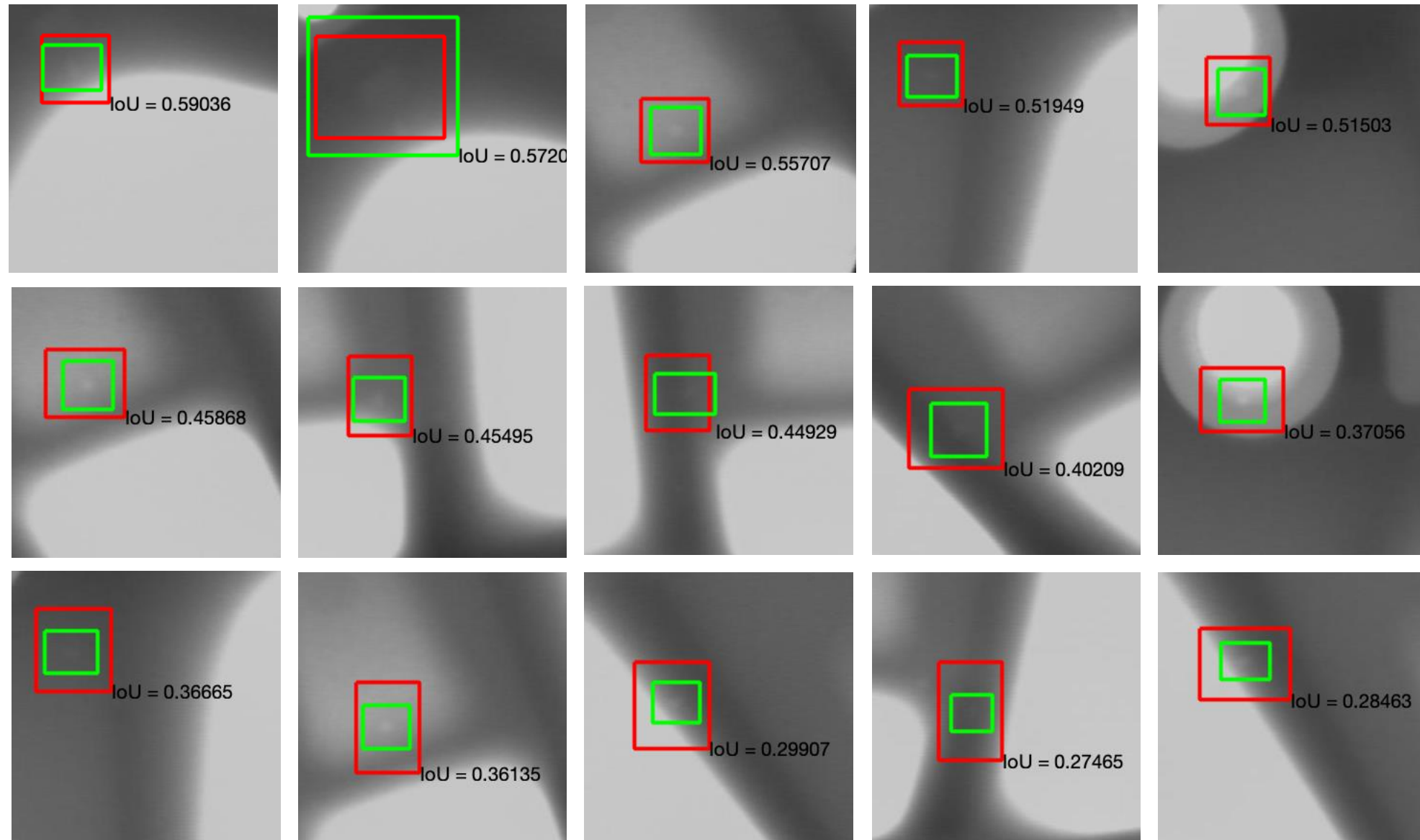
True Positive if $\text{IoU} > \alpha$



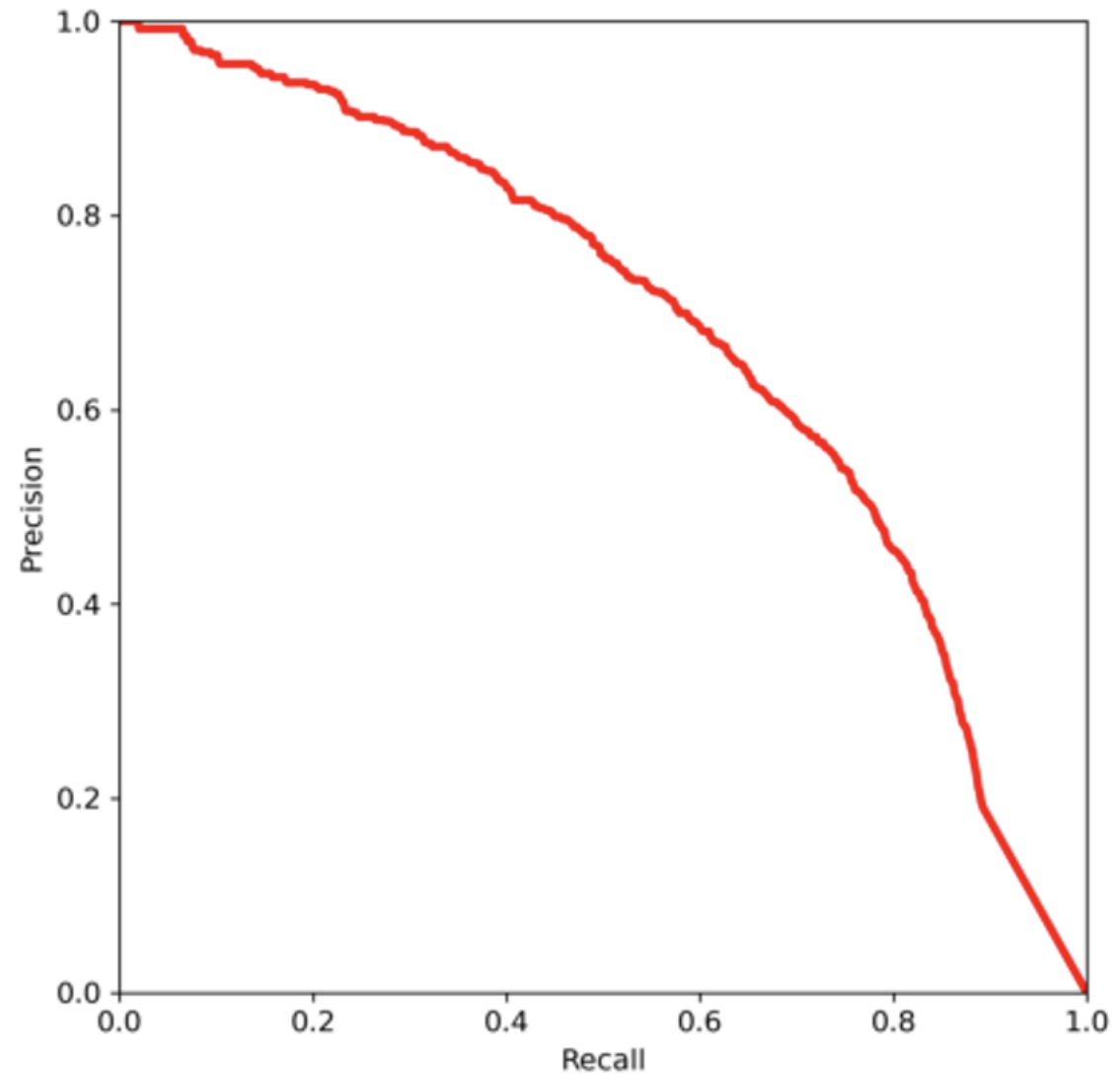
$$\text{IoU} = \frac{\text{area}(\text{GT} \cap \text{DT})}{\text{area}(\text{GT} \cup \text{DT})} > \alpha$$

In our experiments,
we use

$$\alpha = 0.25$$



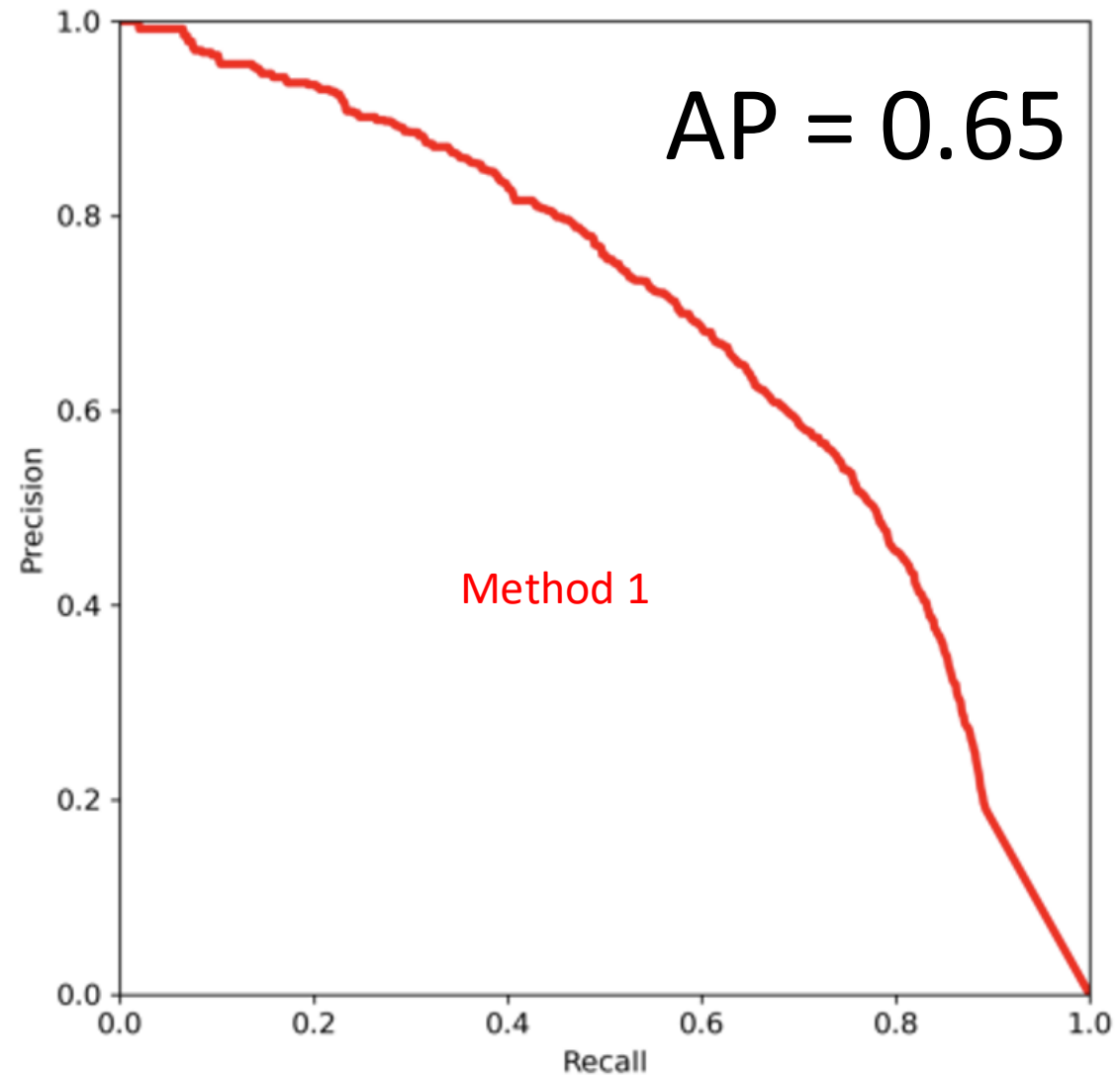
Precision-Recall Curve



Precision-Recall Curve

Average Precision (AP) is
the area under the curve

the higher the better



Precision-Recall Curve

Average Precision (AP) is
the area under the curve

the higher the better

